API Manual Testing Interview Question Bank

Deduplicated, ranked by importance and grouped by topic
Prepared for a Software Tester with 4+ years of experience

Questions in your original document	**137**
Exact duplicates removed (same question, different wording of punctuation or case)	**2**
Near-duplicates merged (same question asked in different words)	**52**
Unique questions in this PDF	**83**

Section 1: Critical (Must Prepare)	22 questions
Section 2: High Priority	38 questions
Section 3: Medium Priority	20 questions
Section 4: Low Priority / Quick Revision	3 questions

How to Use This Document

Questions are ordered by how likely they are to be asked and how much weight interviewers give them for a tester with 4+ years of experience. Work top to bottom: finish Section 1 before you spend time on Section 3 or 4. Inside each section the questions are grouped by topic, and within each topic the most important question comes first.

A red tag such as **[asked x3 in your bank]** means the same question showed up several times in your original document (in different wording). Repetition is a good signal that it is a frequently asked question.

Section	What it means
Critical (Must Prepare) 22 questions	Asked in almost every API testing interview at 4+ years. Give a precise definition plus a Postman or project example.
High Priority 38 questions	Frequent practical and scenario questions. Answer with your approach: what you send, what you assert and what can go wrong.
Medium Priority 20 questions	Occasionally asked, usually as follow-ups or when your resume mentions the topic.
Low Priority / Quick Revision 3 questions	Niche protocols and rare scenarios. Know the one-line idea.

What Interviewers Look For (4+ Years)

Topic	What the interviewer is testing
API Testing Fundamentals	Do you know what API testing covers and how it differs from UI and unit testing?
REST, SOAP & HTTP	Precise answers on methods, PUT vs PATCH, idempotency, status codes and REST vs SOAP.
Postman	Hands-on use: collections, environments, variables, scripts, chaining and Newman, not just 'I sent a request'.
Response Validation	What you check beyond the status code: schema, headers, data integrity, error handling, timeouts.
Authentication & Security	Basic vs OAuth vs JWT, authentication vs authorization, and basic security and rate-limit checks.
Scenario-Based	Judgement on real situations: no documentation, async responses, dependencies, pagination, caching, concurrency.
API Automation	How you move from Postman to code: RestAssured specifications, logging, JSON validation.
Contract Testing	Consumer-driven contracts (Pact) and how contract testing differs from schema validation.

Note: the HTTP/REST basics (status codes, PUT vs PATCH, idempotency, REST vs SOAP) were repeated many times in your file, so they are merged and ranked highest. Your file's title was 'API Manul Testing' (typo in the original).

Table of Contents

Section 1: Critical (Must Prepare) (22 questions) . . . 5
API Testing Fundamentals (2) . . . 5
REST, SOAP & HTTP Concepts (6) . . . 5
Postman: Usage, Scripting & Chaining (7) . . . 6
Response Validation & Test Design (3) . . . 7
Authentication, Security & Rate Limiting (2) . . . 8
Scenario-Based API Testing (1) . . . 8
API Automation (RestAssured) & Java (1) . . . 8

Section 2: High Priority (38 questions) . . . 9
API Testing Fundamentals (3) . . . 9
REST, SOAP & HTTP Concepts (6) . . . 9
Postman: Usage, Scripting & Chaining (2) . . . 10
Response Validation & Test Design (5) . . . 10
Authentication, Security & Rate Limiting (4) . . . 11
Scenario-Based API Testing (12) . . . 11
API Automation (RestAssured) & Java (3) . . . 13
API Contract Testing (3) . . . 13

Section 3: Medium Priority (20 questions) . . . 14
API Testing Fundamentals (3) . . . 14
REST, SOAP & HTTP Concepts (1) . . . 14
Postman: Usage, Scripting & Chaining (4) . . . 14
Response Validation & Test Design (1) . . . 15
Authentication, Security & Rate Limiting (1) . . . 15
Scenario-Based API Testing (7) . . . 15
API Contract Testing (3) . . . 16

Section 4: Low Priority / Quick Revision (3 questions) . . . 17
Authentication, Security & Rate Limiting (1) . . . 17
Scenario-Based API Testing (2) . . . 17

Topic Finder (all sections for one topic)

Use this if you want to study one topic at a time. Each number links to that topic's questions inside a section.

Topic	S1	S2	S3	S4	Total
API Testing Fundamentals	2 Qs, p.5	3 Qs, p.9	3 Qs, p.14	-	**8**
REST, SOAP & HTTP Concepts	6 Qs, p.5	6 Qs, p.9	1 Qs, p.14	-	**13**
Postman: Usage, Scripting & Chaining	7 Qs, p.6	2 Qs, p.10	4 Qs, p.14	-	**13**
Response Validation & Test Design	3 Qs, p.7	5 Qs, p.10	1 Qs, p.15	-	**9**
Authentication, Security & Rate Limiting	2 Qs, p.8	4 Qs, p.11	1 Qs, p.15	1 Qs, p.17	**8**
Scenario-Based API Testing	1 Qs, p.8	12 Qs, p.11	7 Qs, p.15	2 Qs, p.17	**22**
API Automation (RestAssured) & Java	1 Qs, p.8	3 Qs, p.13	-	-	**4**
API Contract Testing	-	3 Qs, p.13	3 Qs, p.16	-	**6**

S1 = Critical, S2 = High, S3 = Medium, S4 = Low priority.

Section 1: Critical (Must Prepare) (22 questions)

Asked in almost every API testing interview at 4+ years. Give a precise definition plus a Postman or project example.

API Testing Fundamentals (2)

1. What is API testing and why is it important? **[asked x3 in your bank]**

   Answer: API testing checks the business-logic layer directly by sending requests and validating responses (status, data, headers, performance, security) without the UI. It is important because it finds defects early, runs faster, is more stable than UI tests and covers logic the UI may hide.

   Example: Test POST /orders directly to confirm the order is created with correct totals, before the checkout UI is ready.

2. What challenges have you faced in API testing?

   Answer: Common challenges: missing or outdated documentation, frequently changing contracts, dependent or unavailable services, token expiry, test data setup/cleanup, async flows and unstable environments. Mention how you solved one (mocks, Swagger/traffic capture, setup scripts).

   Example: Payment sandbox was down, so I used a Postman mock server to keep order tests running.

REST, SOAP & HTTP Concepts (6)

3. Explain the HTTP methods used in API testing (GET, POST, PUT, PATCH, DELETE). **[asked x3 in your bank]**

   Answer: GET reads data, POST creates, PUT replaces a resource completely, PATCH updates part of it, DELETE removes it.

   Example: GET /products/101, POST /orders, PUT /users/7, PATCH /orders/1001 (status), DELETE /cart/items/5.

```
GET    /products/101
POST   /orders           { "productId": 101, "qty": 2 }
PATCH  /orders/1001      { "status": "CANCELLED" }
DELETE /cart/items/5
```

4. What is the difference between PUT and PATCH? **[asked x4 in your bank]**

   Answer: PUT replaces the whole resource (missing fields are reset or removed). PATCH changes only the fields sent. PUT is idempotent; PATCH may or may not be.

   Example: PUT /users/7 with all profile fields vs PATCH /users/7 with only the new phone number.

```
PUT   /users/7  { "name": "Asha", "email": "a@x.com", "phone": "999" }   // full object
PATCH /users/7  { "phone": "999" }                                        // only phone
```

5. What is idempotency? Which HTTP methods are idempotent? **[asked x3 in your bank]**

   Answer: An operation is idempotent when repeating it gives the same result as doing it once. GET, PUT, DELETE, HEAD and OPTIONS are idempotent; POST and usually PATCH are not.

   Example: Calling DELETE /cart/items/5 twice leaves the cart the same; calling POST /orders twice creates two orders.

6. Explain HTTP status codes, their categories and the ones you commonly use. **[asked x5 in your bank]**

   Answer: 1xx informational, 2xx success, 3xx redirection, 4xx client error, 5xx server error. Common: 200 OK, 201 Created, 204 No Content, 400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found, 409 Conflict, 422 Unprocessable, 429 Too Many Requests, 500, 502, 503.

   Example: POST /orders returns 201; wrong token 401; another user's order 403; unknown order 404; duplicate coupon 409.

7. What is a REST API and what are its principles?

> **Answer:** REST is an architectural style using HTTP and resources. Principles: client-server, stateless, cacheable, uniform interface, layered system (and optional code on demand). Resources are identified by URIs and manipulated with HTTP methods.
>
> **Example:** /orders/1001 is a resource; GET reads it, DELETE cancels it; every request carries its own token.

8. What is the difference between SOAP and REST APIs? How do you test SOAP APIs? **[asked x6 in your bank]**

> **Answer:** SOAP is a strict XML protocol with WSDL contract and WS-Security. REST is a lightweight style that supports JSON/XML and HTTP methods. To test SOAP, send an XML envelope with Content-Type text/xml and a SOAPAction header (SoapUI or Postman) and validate against the WSDL/XSD.
>
> **Example:** Legacy payment gateway uses SOAP; the order service uses REST.

```
POST /PaymentService
Content-Type: text/xml; charset=utf-8
SOAPAction: "ProcessPayment"

<soapenv:Envelope xmlns:soapenv="http://schemas.xmlsoap.org/soap/envelope/">
  <soapenv:Body><ProcessPayment><amount>999</amount></ProcessPayment></soapenv:Body>
</soapenv:Envelope>
```

Postman: Usage, Scripting & Chaining (7)

9. What is Postman and why is it widely used in API testing? **[asked x2 in your bank]**

> **Answer:** Postman is an API client to build, send, test and automate requests. It is popular for its simple UI, collections, environments, scripting, collection runner/Newman, mock servers and team collaboration.
>
> **Example:** I keep the shop's cart, checkout and payment API requests in one Postman collection.

10. What are Collections and Environments in Postman? How do you import/export them? **[asked x4 in your bank]**

> **Answer:** A Collection is a group of related requests (with folders, scripts and variables). An Environment is a set of variables per stage (QA/UAT). Export collections and environments as JSON and import them with the Import button or by link.
>
> **Example:** Shop collection with a QA environment holding baseUrl and token.

11. What are variables in Postman (global, environment, collection) and how do you use them?

> **Answer:** Variables avoid hard-coding. Scope from widest to narrowest: global, collection, environment, data, local. Use them as {{baseUrl}} in requests and pm.variables/pm.environment in scripts.
>
> **Example:** {{baseUrl}}/orders/{{orderId}} works in QA and UAT by switching the environment.

```
pm.environment.set("baseUrl", "https://qa.shop.com");
pm.collectionVariables.set("orderId", 1001);
const url = pm.environment.get("baseUrl");
```

12. How do you chain requests: capture a value from one response and pass it to another request? **[asked x3 in your bank]**

> **Answer:** In the Tests script of the first request read the response and save a value into a variable; the next request uses it as {{variable}}.
>
> **Example:** Create an order, save orderId, then call GET /orders/{{orderId}} and POST /payments.

```
const json = pm.response.json();
pm.collectionVariables.set("orderId", json.id);
pm.environment.set("token", json.token);
```

13. What is the difference between the Pre-request Script and the Tests tab?

> **Answer:** Pre-request Script runs before the request is sent (set variables, build data, create tokens or signatures). Tests run after the response arrives (assertions on status, body, headers, time).
>
> **Example:** Pre-request generates a random email; Tests verify 201 and the returned id.

```
// Pre-request
pm.variables.set("email", "user" + Date.now() + "@test.com");
// Tests
pm.test("Created", () => pm.response.to.have.status(201));
```

14. How do you automate API tests in Postman and validate responses with scripts? **[asked x2 in your bank]**

> **Answer:** Write assertions in the Tests tab using pm.test and pm.expect, then run the collection with the Collection Runner or Newman (CI).
>
> **Example:** Check status, order total and response time for POST /orders.

```
pm.test("Status is 201", () => pm.response.to.have.status(201));
pm.test("Total is correct", () => pm.expect(pm.response.json().total).to.eql(1998));
pm.test("Fast response", () => pm.expect(pm.response.responseTime).to.be.below(800));
```

15. How do you implement data-driven API testing (Postman data files, parameterization)? **[asked x3 in your bank]**

> **Answer:** Use the Collection Runner with a CSV or JSON data file; columns become variables used in the request as {{username}} or in scripts with pm.iterationData. Each row is one iteration.
>
> **Example:** Run the login request with many valid and invalid user rows from a CSV file.

```
// users.csv: username,password,expectedStatus
const expected = Number(pm.iterationData.get("expectedStatus"));
pm.test("Status matches", () => pm.response.to.have.status(expected));
```

Response Validation & Test Design (3)

16. How do you validate an API response (status code, body, headers, response time)? What are the most common validations? **[asked x2 in your bank]**

> **Answer:** Validate status code, response body (values, types, required fields), headers (Content-Type, caching), response time and schema. Most common: status, key fields, data types and time.
>
> **Example:** Order response: 201, id present, status is PLACED, Content-Type is JSON, time under 1 s.

```
pm.response.to.have.status(201);
pm.response.to.have.header("Content-Type", "application/json; charset=utf-8");
const body = pm.response.json();
pm.expect(body.status).to.eql("PLACED");
```

17. How do you validate the JSON schema? What is schema validation and how do you test if a response schema changes? **[asked x3 in your bank]**

> **Answer:** Schema validation checks that the response structure (fields, types, required keys) matches a JSON Schema. A failing schema check when fields are added/removed/changed shows a breaking change.
>
> **Example:** Product response schema fails if price changes from number to string.

```
const schema = { type: "object", required: ["id", "price"],
  properties: { id: { type: "integer" }, price: { type: "number" } } };
pm.test("Schema is valid", () => pm.response.to.have.jsonSchema(schema));
```

18. How do you test API error handling and error codes? **[asked x2 in your bank]**

> **Answer:** Send invalid, missing, wrong-type and unauthorized input and check the status code, error message body, structure and that no stack trace or sensitive data is leaked.
>
> **Example:** POST /orders with qty = -1 returns 400 and { "error": "qty must be positive" }.

```
pm.test("400 for bad input", () => pm.response.to.have.status(400));
pm.test("Error message present", () => pm.expect(pm.response.json().error).to.be.a("string"));
```

Authentication, Security & Rate Limiting (2)

19. Explain authentication types (Basic, OAuth, Bearer token). How do you handle authentication in Postman? **[asked x3 in your bank]**

> **Answer:** Basic: base64 username:password (use HTTPS). Bearer token: Authorization: Bearer <token> (often JWT). OAuth 2.0: access tokens from an auth server with scopes. In Postman set auth on the Authorization tab or collection, get the token with a login request or OAuth helper, and store it in a variable.
>
> **Example:** Login request stores the token; collection-level Bearer {{token}} is used for cart and order APIs.

```
Authorization: Bearer {{token}}
Authorization: Basic dXNlcjpwYXNz
```

20. What is the difference between Authentication and Authorization?

> **Answer:** Authentication proves who you are (401 if it fails). Authorization decides what you are allowed to do (403 if not permitted).
>
> **Example:** A valid customer token can view own orders (200) but gets 403 on the admin /users endpoint.

Scenario-Based API Testing (1)

21. How do you approach testing a new API when no documentation is available, or the requirements are incomplete and the behaviour changes often? **[asked x2 in your bank]**

> **Answer:** Explore the app to discover the API (browser DevTools, Swagger/OpenAPI, proxy tools), ask developers/BAs, build a Postman collection from what you find, test happy and negative paths, record assumptions and confirm behaviour when it changes.
>
> **Example:** Captured the cart API calls from the browser Network tab and turned them into a Postman collection.

API Automation (RestAssured) & Java (1)

22. API automation: RestAssured vs Postman tests. When do you use which?

> **Answer:** Postman is great for quick exploratory/manual testing and simple automated checks. RestAssured (Java) suits large automated suites: code reuse, complex logic, version control, TestNG/CI integration and shared utilities with Selenium.
>
> **Example:** Postman to explore a new payment endpoint; RestAssured regression suite running in Jenkins.

```
given().header("Authorization", "Bearer " + token)
  .when().get("/orders/1001")
  .then().statusCode(200).body("status", equalTo("PLACED"));
```

Section 2: High Priority (38 questions)

Frequent practical and scenario questions. Answer with your approach: what you send, what you assert and what can go wrong.

API Testing Fundamentals (3)

23. What is an API and how does it work? What are the different types of APIs? **[asked x2 in your bank]**

> **Answer:** An API (Application Programming Interface) lets one software talk to another: the client sends a request and the server returns a response. Types: REST, SOAP, GraphQL, gRPC; by access: public, partner, private and composite.
>
> **Example:** The mobile app calls the shop's REST API to get products and place orders.

24. What are the common types of API testing?

> **Answer:** Functional, integration, validation, negative, regression, smoke, security, performance/load, contract and usability/reliability testing.
>
> **Example:** Functional: create order; security: access another user's order; load: product search.

25. What is the difference between smoke, regression and load testing in APIs?

> **Answer:** Smoke: quick check that key endpoints work after a deployment. Regression: re-run existing tests to confirm changes did not break anything. Load: many concurrent requests to measure response time and throughput.
>
> **Example:** Smoke: /health, login, /products. Regression: full order collection. Load: 500 users hitting search.

REST, SOAP & HTTP Concepts (6)

26. What are headers, body and parameters in requests and responses? **[asked x2 in your bank]**

> **Answer:** Headers carry metadata (Content-Type, Authorization, Accept). Body carries the payload (JSON/XML). Parameters filter or identify resources: path (/orders/101) and query (?status=PAID). Responses also have status, headers and body.
>
> **Example:** POST /orders with Content-Type: application/json and a JSON body.

```
GET /orders?status=PAID&page=2
Accept: application/json
```

27. How do you handle query parameters and path parameters? **[asked x3 in your bank]**

> **Answer:** Path parameters identify a specific resource (/products/{id}); query parameters filter, sort or paginate (?category=shoes&sort=price). In Postman use the Params tab and :id path variables.
>
> **Example:** GET /customers/7/orders?status=PAID.

28. What is the difference between URI, URL and endpoint? **[asked x2 in your bank]**

> **Answer:** URI identifies a resource; a URL is a URI that also tells where to find it (protocol and address); an endpoint is the specific URL where the API accepts requests.
>
> **Example:** Endpoint: https://api.shop.com/v1/orders (a URL); resource URI: /orders/1001.

29. What is JSON and why is it preferred over XML? **[asked x2 in your bank]**

> **Answer:** JSON is lightweight, readable and easy to parse with native types and arrays, so payloads are smaller and faster. XML is more verbose but supports namespaces, attributes and strong schemas.
>
> **Example:** The shop's mobile API uses JSON; the legacy payment service uses XML.

30. How do you test APIs with different content types (JSON, XML, form data)? Raw vs form-data body? **[asked x2 in your bank]**

> **Answer:** JSON/XML use a raw body with Content-Type application/json or application/xml. form-data (multipart) sends key-value pairs and files. x-www-form-urlencoded sends simple key=value forms. Raw sends the text as written.
>
> **Example:** Upload a product image with form-data; create an order with raw JSON.

31. What is API versioning and what are the common strategies?

> **Answer:** Versioning keeps old clients working while the API changes. Strategies: URI (/v1/), header, query parameter, or media type.
>
> **Example:** /v1/orders keeps working for the old app while /v2/orders adds new fields.

Postman: Usage, Scripting & Chaining (2)

32. How do you run Postman tests from the command line (Newman) and in CI?

> **Answer:** Export the collection and environment and run with Newman (npm install -g newman). In CI (Jenkins/GitHub Actions) run the same command and publish the report.
>
> **Example:** Nightly Jenkins job runs the shop collection against QA.

```
newman run shop.postman_collection.json -e qa.postman_environment.json \
  -d users.csv -r cli,htmlextra
```

33. Explain the structure of a request in Postman.

> **Answer:** A Postman request has: method, URL, params, authorization, headers, body, pre-request script, tests and settings.
>
> **Example:** POST {{baseUrl}}/orders with Bearer auth, JSON body and a test for 201.

Response Validation & Test Design (5)

34. How do you handle timeouts, retries and error scenarios in API testing? **[asked x2 in your bank]**

> **Answer:** Set request timeouts, simulate slow or failing services, check status codes such as 408/503/504 and retry behaviour (with backoff and idempotency), and verify error messages and recovery.
>
> **Example:** Delay the payment service and check the order API returns 504 and does not create a duplicate order on retry.

35. How do you validate data integrity in API testing?

> **Answer:** Check that data sent is stored and returned correctly: compare the POST body with the GET response and the database, check types, relationships and no data loss or duplicates.
>
> **Example:** After POST /orders, confirm order amount in the API, the DB and the orders list are the same.

36. How do you handle dynamic payloads?

> **Answer:** Generate values at runtime using Postman dynamic variables, pre-request scripts or templates (POJOs/maps in RestAssured), so payloads do not contain hard-coded data.
>
> **Example:** Unique email and timestamp for every registration request.

```
{
  "email": "{{$randomEmail}}",
  "orderRef": "{{$guid}}",
  "createdAt": "{{$isoTimestamp}}"
}
```

37. How do you ensure API backward compatibility?

> **Answer:** Do not remove or rename fields or change types; add only optional fields; keep old versions running; run the old regression/consumer contract tests against the new version.
>
> **Example:** Adding discountCode as optional is safe; renaming price to amount breaks the mobile app.

38. What are API testing best practices?

> **Answer:** Cover happy and negative paths; validate status, body, headers, schema and time; keep tests independent; use environments and variables; clean test data; automate in CI; keep collections version-controlled.
>
> **Example:** Each order test creates its own cart and deletes it afterwards.

Authentication, Security & Rate Limiting (4)

39. How do you test API authentication (OAuth, JWT)? How do you handle tokens, OAuth and CORS in Postman? **[asked x2 in your bank]**

> **Answer:** Check valid token (200), missing/invalid/expired/tampered token (401), wrong role (403). In Postman store tokens in variables, refresh them in a pre-request script and use OAuth helper. CORS is enforced by browsers, so test it by sending an OPTIONS request with an Origin header and checking Access-Control-Allow-* headers.
>
> **Example:** Expired JWT on GET /orders must return 401.

```
Authorization: Bearer <expired-or-tampered-jwt>     -> expect 401
OPTIONS /orders
Origin: https://evil.com                            -> check Access-Control-Allow-Origin
```

40. What is the difference between OAuth 1.0 and OAuth 2.0? When do you use each? Write a sample.

> **Answer:** OAuth 1.0 signs every request (HMAC) and is complex; OAuth 2.0 uses bearer access tokens over HTTPS with scopes and grant types (authorization code, client credentials) and is the standard. Use OAuth 2.0 for new APIs; OAuth 1.0 only for legacy systems.
>
> **Example:** Client-credentials token to call the order API.

```
POST /oauth/token
grant_type=client_credentials&client_id=shop-app&client_secret=***

GET /orders
Authorization: Bearer eyJhbGciOi...
```

41. How do you test API security (in Postman)? **[asked x2 in your bank]**

> **Answer:** Test authentication bypass, expired/tampered tokens, role access and IDOR (changing ids), injection payloads, sensitive data exposure, HTTPS only, rate limiting, CORS and verbose error leakage.
>
> **Example:** Change orderId in /orders/1002 using customer A's token: expect 403 or 404, not B's order.

42. How do you test API rate limiting (throttling)? **[asked x3 in your bank]**

> **Answer:** Send many requests quickly (Collection Runner/Newman/JMeter) and check the API returns 429 Too Many Requests with Retry-After and X-RateLimit headers, and that access resumes after the window.
>
> **Example:** More than 100 search calls per minute returns 429.

```
HTTP/1.1 429 Too Many Requests
Retry-After: 30
X-RateLimit-Limit: 100
X-RateLimit-Remaining: 0
```

Scenario-Based API Testing (12)

43. How do you test API pagination? **[asked x3 in your bank]**

> **Answer:** Test page/size (or limit/offset or cursor) parameters: first/middle/last page, beyond the last page, invalid values, total count, no duplicates or gaps across pages and default page size.
>
> **Example:** GET /products?page=2&size=20 returns 20 items, correct totals and different items than page 1.

```
GET /products?page=1&size=20     -> 20 items, totalPages = N
GET /products?page=999&size=20   -> empty list (or 404)
GET /products?page=0&size=-5     -> 400
```

44. How do you test sorting and filtering in APIs?

> **Answer:** Verify sort order for ascending/descending and for each field, filter combinations, no-match filters, invalid fields and that sorting is consistent across pages.
>
> **Example:** GET /products?category=shoes&sort=price,asc returns only shoes in rising price.

```
const prices = pm.response.json().items.map(i => i.price);
pm.expect(prices).to.eql([...prices].sort((a, b) => a - b));
```

45. How do you test APIs that return asynchronous responses? **[asked x2 in your bank]**

> **Answer:** Expect 202 Accepted with a job id or status URL, poll the status endpoint until completed (with a limit), then verify the final result; also test failure, timeout and callbacks/webhooks.
>
> **Example:** Report export returns 202 and a statusUrl; poll until status is COMPLETED, then download.

46. How do you test APIs that depend on other services (and test APIs in a microservices architecture)? **[asked x2 in your bank]**

> **Answer:** Test each service alone, then integration; use mocks/stubs for unavailable dependencies, verify data flow and contract between services and test failure handling between them.
>
> **Example:** Order service tested with a mocked payment service and then end to end.

47. How do you test APIs with third-party integrations?

> **Answer:** Use the provider's sandbox/test keys, mock the third party for failures/timeouts, validate request/response mapping, error handling, rate limits and fallback behaviour.
>
> **Example:** Payment gateway sandbox with a test card for success and decline scenarios.

48. How do you validate API caching behaviour? **[asked x2 in your bank]**

> **Answer:** Check Cache-Control, ETag and Last-Modified headers, send If-None-Match and expect 304, and verify data refreshes after an update or when the TTL expires.
>
> **Example:** GET /products/101 returns an ETag; after a price change the new ETag and price are returned.

```
GET /products/101
If-None-Match: "abc123"      -> 304 Not Modified (unchanged)
```

49. How do you test APIs under failure scenarios?

> **Answer:** Simulate downstream failures, timeouts, DB down and bad input; expect graceful 5xx/503 with a clear message, retries/circuit breaker, and no data corruption or partial records.
>
> **Example:** Stock service down: order API returns 503 and no order is saved.

50. How do you test API concurrency issues?

> **Answer:** Send simultaneous requests (JMeter, Newman parallel runs, RestAssured with threads) to the same resource and check data stays correct: no overselling, no duplicate orders, proper locking or 409 conflicts.
>
> **Example:** Two users buy the last item at once: one succeeds, the other gets 409/out of stock.

51. How do you test the performance of an API (Postman, product search API)? **[asked x2 in your bank]**

> **Answer:** In Postman check response time per request and run the collection with several iterations; for real load use JMeter, k6 or Gatling and track average, p95, throughput and error rate.
>
> **Example:** Product search API must respond under 500 ms at 200 users.

```
pm.test("Under 500 ms", () => pm.expect(pm.response.responseTime).to.be.below(500));
```

52. What if the API response is slow but the UI has no timeout handling?

> **Answer:** Delay the API response (mock, proxy or throttling) and observe the UI: it should show a loader, timeout message and retry option. If it hangs, raise a defect and verify API timeout behaviour (504).
>
> **Example:** Slow product API leaves the page blank forever: defect to add a timeout and error message.

53. How do you test APIs that behave differently for different users (roles)?

> **Answer:** Get tokens for each role and test allowed and forbidden calls (200 vs 403), data scoping (users see only their own data) and role-based fields.
>
> **Example:** Customer cannot call DELETE /products; admin can.

54. How do you test APIs with file uploads in Postman?

> **Answer:** In Postman choose Body > form-data, set the key type to File and select the file. Verify success (201), stored file details, size/type validation, empty, large and invalid files.
>
> **Example:** Upload a product image of 2 MB: expect 201; upload an .exe: expect 400/415.

```
POST /products/101/image
Content-Type: multipart/form-data
file: product.png
```

API Automation (RestAssured) & Java (3)

55. Explain RequestSpecification and ResponseSpecification in RestAssured.

> **Answer:** RequestSpecification holds reusable request settings (base URI, headers, auth, content type). ResponseSpecification holds reusable expected checks (status, content type, time). Build them with RequestSpecBuilder/ResponseSpecBuilder.
>
> **Example:** A common spec adds the base URL and token to every shop API test.

```
RequestSpecification req = new RequestSpecBuilder().setBaseUri("https://qa.shop.com")
    .addHeader("Authorization", "Bearer " + token).setContentType(ContentType.JSON).build();
ResponseSpecification res = new ResponseSpecBuilder().expectStatusCode(200)
    .expectContentType(ContentType.JSON).build();
given().spec(req).get("/orders/1001").then().spec(res);
```

56. How do you log request and response details?

> **Answer:** Use log().all() on the request and response, log().ifValidationFails() to log only on failure, or logging filters.
>
> **Example:** Log the full request when the order test fails in CI.

```
given().log().all()
  .when().get("/orders/1001")
  .then().log().ifValidationFails().statusCode(200);
```

57. How do you validate a JSON response using Java while testing APIs?

> **Answer:** Use RestAssured body() with Hamcrest matchers or jsonPath() to read values, or map the response to a POJO with Jackson; use matchesJsonSchemaInClasspath for schema checks.
>
> **Example:** Check order id, status and item list from GET /orders/1001.

```
given().get("/orders/1001").then().statusCode(200)
  .body("id", equalTo(1001)).body("items.size()", greaterThan(0))
  .body(matchesJsonSchemaInClasspath("order-schema.json"));
```

API Contract Testing (3)

58. What is API contract testing and why is it important? **[asked x2 in your bank]**

> **Answer:** Contract testing verifies that an API provider and its consumers agree on request and response expectations, so changes that break a consumer are caught early without full end-to-end tests.
>
> **Example:** The mobile app contract with the order API fails the build if a field it uses is removed.

59. What tools are used for API contract testing? What is Pact and how does it work? **[asked x2 in your bank]**

> **Answer:** Tools: Pact, Spring Cloud Contract, Specmatic, Dredd, Postman schema checks. Pact is consumer-driven: the consumer test creates a pact file (contract), the provider verifies it, and a Pact Broker stores contracts and results.
>
> **Example:** Cart service (consumer) publishes its contract; the product service (provider) verifies it in CI.

60. What is the difference between schema validation and API contract testing?

> **Answer:** Schema validation checks that one response matches a structure. Contract testing checks the agreed interactions (request, response, status, headers, behaviour) between a specific consumer and provider and is driven by consumer needs.
>
> **Example:** Schema: price is a number. Contract: the cart service expects price and stock in /products/101.

Section 3: Medium Priority (20 questions)

Occasionally asked, usually as follow-ups or when your resume mentions the topic.

API Testing Fundamentals (3)

61. What is the difference between API testing and unit testing?

> **Answer:** Unit tests check small pieces of code in isolation (by developers, often with mocks). API tests check the endpoint behaviour end to end over HTTP across layers (often by QA).
>
> **Example:** Unit test: discount function. API test: POST /orders applies the discount and saves the order.

62. What is the difference between functional and non-functional API testing?

> **Answer:** Functional testing checks correct behaviour (CRUD, validations, business rules). Non-functional checks performance, security, reliability, scalability and usability.
>
> **Example:** Functional: coupon reduces the total. Non-functional: coupon API handles 500 requests per second.

63. What tools are commonly used for API testing?

> **Answer:** Postman, SoapUI, RestAssured, JMeter, Insomnia, Karate, Swagger UI, curl, Newman and Pact (contracts).
>
> **Example:** Postman for manual testing, RestAssured for automation, JMeter for load.

REST, SOAP & HTTP Concepts (1)

64. What are the advantages of RESTful APIs?

> **Answer:** RESTful APIs are simple, lightweight and stateless (easy to scale), cacheable, platform independent, support many formats and use standard HTTP so they are easy to test.
>
> **Example:** The same REST API serves the web store, mobile app and partners.

Postman: Usage, Scripting & Chaining (4)

65. How do you integrate Postman into your project?

> **Answer:** Store collections in Git or a shared workspace, use environments per stage, run with Newman in CI, publish reports and add monitors for scheduled checks.
>
> **Example:** Pull request pipeline runs the Postman smoke collection on every build.

66. How do you keep Postman environments in sync across teams?

> **Answer:** Use shared workspaces or version control, keep environment-specific values as variables, store secrets in current values or a vault (not shared), and sync via export/import or Git.
>
> **Example:** QA and dev share the QA environment file; each person keeps their own token locally.

67. What are common mistakes in Postman testing?

> **Answer:** Common mistakes: checking only status codes, hard-coded values, no negative tests, test order dependency, secrets in shared collections, not cleaning up data and not versioning collections.
>
> **Example:** A hard-coded orderId fails the next day because the order was deleted.

68. How do you mock APIs in Postman?

> **Answer:** Create a mock server from a collection with saved example responses; use its mock URL in tests to simulate unavailable or failing services.
>
> **Example:** Mock the payment API to return a failure response for checkout negative tests.

Response Validation & Test Design (1)

69. How do you test APIs with complex JSON structures?

> **Answer:** Break the response into parts and verify key fields, nested objects and arrays (size, types, each element), use JSONPath/jsonPath queries and a schema, and handle optional fields.
>
> **Example:** Check each item in order.items has sku, qty and price.

```
const items = pm.response.json().items;
pm.expect(items).to.be.an("array").that.is.not.empty;
items.forEach(i => pm.expect(i).to.have.all.keys("sku", "qty", "price"));
```

Authentication, Security & Rate Limiting (1)

70. How do you test data encryption in API/DB?

> **Answer:** Check the API uses HTTPS/TLS (HTTP is rejected or redirected, HSTS header), sensitive fields are not returned in plain text and the database stores them encrypted/hashed.
>
> **Example:** Passwords and card numbers are never visible in responses or the DB in plain text.

```
Strict-Transport-Security: max-age=31536000
// Verify in the DB that password_hash and card_enc are not readable text
```

Scenario-Based API Testing (7)

71. How do you test versioned APIs?

> **Answer:** Test each version separately, confirm old versions still work, new fields exist only where designed, deprecation headers and invalid-version behaviour; run the same collection per version with an environment variable.
>
> **Example:** Run the order collection for /v1 and /v2 and compare results.

72. How do you test webhooks in Postman?

> **Answer:** Register a receiver URL (webhook.site, a mock server or an ngrok tunnel), trigger the event, then verify the callback arrives with the correct payload, signature/headers, retries and no duplicates.
>
> **Example:** Order shipped event posts to the registered URL with status SHIPPED.

73. How do you test GraphQL APIs in Postman?

> **Answer:** Send POST /graphql with a query and variables (Postman GraphQL body). Check the data and the errors array (status may still be 200), and test invalid fields, nested queries, mutations and auth.
>
> **Example:** Query a product with name, price and reviews in one call.

```
query ($id: ID!) { product(id: $id) { name price reviews { rating } } }
// variables: { "id": "101" }
```

74. How do you test APIs with file downloads?

> **Answer:** Send the GET request and check status, Content-Type, Content-Disposition and Content-Length, save the response to a file, compare size or checksum and test invalid ids, auth and large files.
>
> **Example:** Download invoice PDF for order 1001 and check the file is a valid PDF.

```
Content-Type: application/pdf
Content-Disposition: attachment; filename="invoice-1001.pdf"
```

75. How do you test an API migration from monolith to microservices?

> **Answer:** Run the same requests against the monolith and the new services and compare responses (parity), test contracts, data consistency, gateway routing, auth propagation, failure handling and performance, with a rollback plan.
>
> **Example:** The order endpoint returns identical results before and after moving to the order microservice.

76. How do you test APIs with scheduled jobs?

> **Answer:** Trigger the job manually or with a short schedule in the test environment, wait/poll until it finishes, then verify its results, failure and retry handling, time zone and that it does not run twice.
>
> **Example:** Nightly job that cancels unpaid orders after 24 hours.

77. How do you test localization in APIs?

> **Answer:** Send the Accept-Language header and check translated messages, date/number/currency formats, default fallback for unsupported languages and UTF-8 characters in data.
>
> **Example:** Accept-Language: hi returns the error message in Hindi.

```
GET /products/101
Accept-Language: hi-IN
```

API Contract Testing (3)

78. What are the key components of an API contract?

> **Answer:** An API contract defines endpoints and methods, request parameters/headers/body, response schema, status codes, error format, authentication, versioning and constraints, usually written in OpenAPI.
>
> **Example:** OpenAPI file describing /orders request, 201 response and 400 error format.

79. How do you ensure backward compatibility in API contract testing?

> **Answer:** Verify every new provider version against all existing consumer contracts (for example Pact can-i-deploy), allow additive changes only, deprecate gradually and run contract checks in CI.
>
> **Example:** Adding an optional field passes; removing a field used by the app fails.

80. What are the challenges of API contract testing?

> **Answer:** Challenges: maintaining contracts, many consumers, provider states and test data, learning the tools, cross-team coordination, versioning and setting up a broker.
>
> **Example:** Different teams need to agree on when to update shared contracts.

Section 4: Low Priority / Quick Revision (3 questions)

Niche protocols and rare scenarios. Know the one-line idea.

Authentication, Security & Rate Limiting (1)

81. How do you test APIs that require digital signatures?

> **Answer:** Create the signature (for example HMAC-SHA256) from the payload and secret in a pre-request script and send it in a header. Test valid signature, tampered body, wrong key, missing signature and replay with an old timestamp.
>
> **Example:** Payment callback must be rejected if the signature does not match.

```
const body = pm.request.body.raw;
const sig = CryptoJS.HmacSHA256(body, pm.environment.get("secret")).toString();
pm.request.headers.add({ key: "X-Signature", value: sig });
```

Scenario-Based API Testing (2)

82. How do you test gRPC APIs in Postman?

> **Answer:** In Postman create a gRPC request, enter the server URL, import the .proto file or use server reflection, select the method, send a JSON message and check the response and gRPC status (OK, NOT_FOUND, etc.); streaming calls are also supported.
>
> **Example:** Call the product service GetProduct method with id 101.

83. How do you test an API that returns streaming data?

> **Answer:** Use a client that supports streaming (curl -N for server-sent events, WebSocket tools), then verify chunks arrive in order with the right format, the connection stays open and closes properly, and reconnection works.
>
> **Example:** Live order-status updates sent as a stream.

```
curl -N -H "Accept: text/event-stream" https://api.shop.com/orders/1001/stream
```